LP3 Practical Exam – All Codes

Compact two-column layout, similar to the uploaded original

1. Uber Fare (ML)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error

df = pd.read_csv("uber.csv")
print(df.head())
df = df.drop(["Unnamed: 0", "key", "pickup_datetime"], axis=1).dropna()
df = df[(df["fare_amount"] > 0) & (df["passenger_count"] > 0)]

def distance(lat1, lon1, lat2, lon2):
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    a = (np.sin((lat2-lat1)/2)**2 +
         np.cos(lat1)*np.cos(lat2)*np.sin((lon2-lon1)/2)**2)
    return 6371 * 2 * np.arcsin(np.sqrt(a))

df["distance_km"] = distance(df["pickup_latitude"],
    df["pickup_longitude"], df["dropoff_latitude"], df["dropoff_longitude"])
df = df[df["distance_km"] > 0]

sns.boxplot(x=df["fare_amount"])
plt.title("Fare Amount Outliers")
plt.show()

def remove_outliers(data, col):
    q1, q3 = data[col].quantile(0.25), data[col].quantile(0.75)
    iqr = q3 - q1
    return data[(data[col] >= q1 - 1.5*iqr) &
                (data[col] <= q3 + 1.5*iqr)]

df = remove_outliers(df, "fare_amount")
df = remove_outliers(df, "distance_km")
print(df.corr(numeric_only=True))
sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.show()

X = df[["distance_km", "passenger_count"]]
y = df["fare_amount"]
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)

lr = LinearRegression().fit(X_train, y_train)
pred_lr = lr.predict(X_test)
rf = RandomForestRegressor(n_estimators=50, random_state=42,
    n_jobs=-1).fit(X_train, y_train)
pred_rf = rf.predict(X_test)

print("Linear Regression R2:", r2_score(y_test, pred_lr))
print("Linear Regression RMSE:", np.sqrt(mean_squared_error(y_test, pred_lr)))
print("Random Forest R2:", r2_score(y_test, pred_rf))
print("Random Forest RMSE:", np.sqrt(mean_squared_error(y_test, pred_rf)))
```

2. K-Means (ML)

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

data = pd.read_csv("sales_data_sample.csv", encoding="ISO-8859-1")
print(data.head())
X = data.select_dtypes(include="number")
X = X.fillna(X.mean())

X_scaled = StandardScaler().fit_transform(X)
wcss = [KMeans(n_clusters=k, random_state=42,
    n_init=10).fit(X_scaled).inertia_ for k in range(1, 11)]

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker="o")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.grid(True)
plt.show()

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
data["Cluster"] = kmeans.fit_predict(X_scaled)
print(data[["Cluster"]].head(10))
print(data["Cluster"].value_counts())
data.to_csv("sales_data_clustered.csv", index=False)
```

3. KNN Diabetes (ML)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (confusion_matrix, accuracy_score,
    precision_score, recall_score)

df = pd.read_csv("diabetes.csv")
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
y_pred = knn.predict(X_test)
acc = accuracy_score(y_test, y_pred)

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", acc)
print("Error Rate:", 1 - acc)
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
```

4. Churn Neural Network (ML)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Dropout

df = pd.read_csv("Churn_Modelling.csv")
X = df.drop(["RowNumber", "CustomerId", "Surname", "Exited"], axis=1)
y = df["Exited"]
X = pd.get_dummies(X, columns=["Geography", "Gender"],
    drop_first=True, dtype=int)
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

model = Sequential([Input(shape=(X_train.shape[1],)),
    Dense(6, activation="relu"),
    Dense(1, activation="sigmoid")])
model.compile(loss="binary_crossentropy", optimizer="adam",
    metrics=["accuracy"])
model.fit(X_train, y_train, epochs=20, batch_size=32, verbose=0)
pred = (model.predict(X_test) > 0.5).astype(int)
print("Basic Model Accuracy:", accuracy_score(y_test, pred))

model2 = Sequential([Input(shape=(X_train.shape[1],)),
    Dense(16, activation="relu"),
    Dropout(0.2),
    Dense(8, activation="relu"),
    Dense(1, activation="sigmoid")])
model2.compile(loss="binary_crossentropy", optimizer="adam",
    metrics=["accuracy"])
model2.fit(X_train, y_train, epochs=50, batch_size=32, verbose=0)
pred2 = (model2.predict(X_test) > 0.5).astype(int)
print("Improved Model Accuracy:", accuracy_score(y_test, pred2))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred2))
```

5. Spam KNN + SVM (ML)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.read_csv("emails.csv")
print(df.head())
df = df.dropna().drop("Email No.", axis=1)
X = df.drop("Prediction", axis=1)
y = df["Prediction"]
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)

p1 = KNeighborsClassifier(n_neighbors=7).fit(X_train,
    y_train).predict(X_test)
print("KNN Accuracy:", accuracy_score(y_test, p1))
print("KNN Confusion Matrix:\n", confusion_matrix(y_test, p1))

p2 = SVC(kernel="linear", C=1).fit(X_train, y_train).predict(X_test)
print("SVM Accuracy:", accuracy_score(y_test, p2))
print("SVM Confusion Matrix:\n", confusion_matrix(y_test, p2))
```

7. Bank Contract (Solidity)

```
pragma solidity ^0.8.0;

contract Bank {
    mapping(address => uint) balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw(uint amount) public {
        require(balances[msg.sender] >= amount, "Not enough balance");
        balances[msg.sender] -= amount;
        payable(msg.sender).transfer(amount);
    }

    function showBalance() public view returns (uint) {
        return balances[msg.sender];
    }
}
```

8. Student Data (Solidity)

```
pragma solidity ^0.8.0;

contract StudentData {
    struct Student { uint rollNo; string name; uint marks; }
    Student[] public students;

    function addStudent(uint _rollNo, string memory _name,
        uint _marks) public {
        students.push(Student(_rollNo, _name, _marks));
    }

    function getStudent(uint index) public view returns (uint,
        string memory, uint) {
        Student memory s = students[index];
        return (s.rollNo, s.name, s.marks);
    }

    function getCount() public view returns (uint) {
        return students.length;
    }

    fallback() external payable { }
    receive() external payable { }
}
```

10. Fibonacci (DAA, input)

```
n = int(input("Enter number of terms: "))

def fib_rec(k):
    return k if k <= 1 else fib_rec(k-1) + fib_rec(k-2)

a, b = 0, 1
series = []
for i in range(n):
    series.append(a)
    a, b = b, a + b

print("Non-recursive:", series)
print("Recursive:", [fib_rec(i) for i in range(n)])
```

11. Huffman Encoding (DAA, input)

```
import heapq

class Node:
    def __init__(self, freq, symbol, left=None, right=None):
        self.freq, self.symbol = freq, symbol
        self.left, self.right = left, right

    def __lt__(self, other):
        return self.freq < other.freq

def print_codes(node, code=""):
    if node.left is None and node.right is None:
        print(node.symbol, "->", code)
        return
    print_codes(node.left, code + "0")
    print_codes(node.right, code + "1")

n = int(input("Enter number of characters: "))
heap = []
for i in range(n):
    ch, f = input("Enter character and frequency: ").split()
    heap.append(Node(int(f), ch))

heapq.heapify(heap)
while len(heap) > 1:
    left = heapq.heappop(heap)
    right = heapq.heappop(heap)
    heapq.heappush(heap, Node(left.freq + right.freq,
        left.symbol + right.symbol, left, right))

print("Huffman Codes:")
print_codes(heap[0])
```

12. Fractional Knapsack (DAA, input)

```
n = int(input("Enter number of items: "))
weight, profit = [], []
for i in range(n):
    w, p = map(int,
        input("Enter weight and profit of item %d: " % (i + 1)).split())
    weight.append(w)
    profit.append(p)

capacity = int(input("Enter capacity of bag: "))
items = [(profit[i] / weight[i], weight[i],
    profit[i]) for i in range(n)]
items.sort(reverse=True)

total = 0
for ratio, w, p in items:
    if capacity == 0:
        break
    if w <= capacity:
        total += p
        capacity -= w
    else:
        total += p * capacity / w
        capacity = 0

print("Maximum profit =", total)
```

13. 0-1 Knapsack (DAA, input)

```
n = int(input("Enter number of items: "))
wt, pr = [0], [0]
for i in range(n):
    w, p = map(int,
        input("Enter weight and profit of item %d: " % (i + 1)).split())
    wt.append(w)
    pr.append(p)

W = int(input("Enter capacity of bag: "))
dp = [[0] * (W + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
    for w in range(W + 1):
        if wt[i] <= w:
            dp[i][w] = max(dp[i-1][w], pr[i] + dp[i-1][w - wt[i]])
        else:
            dp[i][w] = dp[i-1][w]

print("Maximum profit =", dp[n][W])
w = W
print("Items taken:", end=" ")
for i in range(n, 0, -1):
    if dp[i][w] != dp[i-1][w]:
        print(i, end=" ")
        w -= wt[i]
print()
```

14. N-Queens (DAA, input)

```
N = int(input("Enter N (e.g. 8): "))
board = [[0] * N for _ in range(N)]

def is_safe(row, col):
    for i in range(row):
        if board[i][col] == 1:
            return False

    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    i, j = row - 1, col + 1
    while i >= 0 and j < N:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1
    return True

def solve(row):
    if row == N:
        return True
    for col in range(N):
        if is_safe(row, col):
            board[row][col] = 1
            if solve(row + 1):
                return True
            board[row][col] = 0
    return False

first = int(input(
    "Column for first queen in row 0 (0 to %d): " % (N - 1)))
board[0][first] = 1
if solve(1):
    print("N-Queens Solution:")
    for r in board:
        print(*r)
else:
    print("No solution with this first queen position")
```